

DAY 19

Retrieval Quality, Ranking, and RAG Evaluation

GPT + AI Agents + RAG + n8n + API Integration + Business Automation

Duration	60 minutes
Level	Intermediate RAG Foundations
Main Project	Evaluate and Tune a Company Knowledge Retriever
Core Transition	From "search returns something" to "we can prove retrieval returns the right evidence"

Day 19 Core Question

How do we measure whether retrieval is finding the **right evidence, in the right order, with the right amount of noise** before asking an LLM to answer?

Prepared for the 30-Day AI Automation Lecture Series

Table of Contents

1. Learning Goals and Schedule
2. Day 18 Review
3. What Retrieval Quality Means
4. Gold Evidence and Evaluation Datasets
5. Recall, Precision, and Hit Rate
6. Ranking Metrics: MRR and nDCG
7. Top-K and Score Thresholds
8. Metadata Filtering and Access Scope
9. Query Rewriting and Decomposition
10. Hybrid Retrieval
11. Reranking
12. Answer-Level RAG Evaluation
13. No-Answer and Negative Tests
14. Retrieval Failure Taxonomy
15. Tuning and Regression Workflow
16. Current OpenAI Vector Store Search Controls
17. Practical Python Evaluation Lab
18. Production Monitoring and Business Design
19. Exercise, Quiz, Homework, and Interview Prep
20. Day 20 Preview and References

Teaching Note

Day 19 is intentionally evaluation-first. A RAG system should be tuned against realistic questions and expected evidence, not against one impressive demo.

1. Learning Goals and 60-Minute Schedule

Students move from building a retrieval pipeline to **measuring and improving** it. The target skill is to diagnose whether failures come from ingestion, chunking, query formulation, filtering, candidate retrieval, ranking, context assembly, or generation.

Learning Objectives

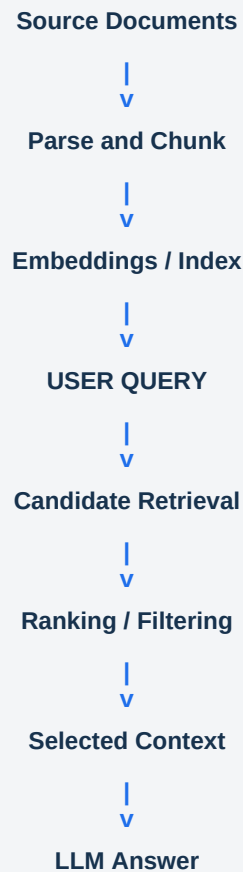
- Explain retrieval relevance at chunk and document level.
- Build a small gold evaluation dataset with expected evidence.
- Calculate Recall@K, Precision@K, Hit@K, and reciprocal rank.
- Understand why ranking position matters, not just whether a relevant chunk appears somewhere.
- Tune top-K and score thresholds using data rather than intuition.
- Use metadata filters to reduce irrelevant candidates and enforce access scope.
- Explain query rewriting, query decomposition, hybrid search, and reranking.
- Separate retrieval quality from answer-generation quality.
- Design no-answer tests and negative controls.
- Create a regression loop for continuous RAG improvement.

Time	Topic
0-5	Day 18 review
5-12	Retrieval quality and gold evidence
12-20	Recall, precision, Hit@K
20-27	Ranking metrics and Top-K
27-34	Thresholds, metadata, query rewriting
34-41	Hybrid search and reranking
41-48	Answer-level RAG evaluation
48-54	Failure analysis and tuning
54-58	Python evaluation lab
58-60	Quiz and homework

2. Day 18 Review: Chunking Creates the Search Units

Day 18 focused on how documents become chunks. Day 19 assumes those chunks already exist and asks whether the retrieval system can find the right ones for realistic user questions.

Where Day 19 operates in RAG



Day 18

Chunk boundaries; overlap; structure; metadata; parent-child retrieval.

Day 19

Evaluation labels; ranking; thresholds; query treatment; failure diagnosis.

Dependency

A poor chunking strategy can cap retrieval performance. Evaluation tells you **that** retrieval is failing; investigation tells you **why**.

3. What Retrieval Quality Means

A retriever is good when it consistently returns evidence that is useful for answering the query, ranks that evidence near the top, excludes unnecessary noise, respects metadata/security scope, and behaves safely when no supporting evidence exists.

Four Dimensions of Retrieval Quality

- Coverage - did we retrieve the required evidence?
- Precision - how much of what we retrieved is actually useful?
- Ordering - did the best evidence appear early?
- Scope - were irrelevant versions, departments, regions, or unauthorized sources excluded?

Example

Query: How many vacation days can roll into next year?

Expected evidence:

"Unused vacation days may carry over up to 5 days."

Retriever A top 3:

1. Vacation carryover clause [relevant]
2. Expense reimbursement [irrelevant]
3. Remote work eligibility [irrelevant]

Retriever B top 3:

1. Vacation policy summary [relevant]
2. Vacation carryover clause [relevant]
3. PTO definitions [partly relevant]

Both systems found something useful, but their quality profiles differ. Evaluation converts this intuition into measurable behavior.

4. Gold Evidence and Evaluation Datasets

Before tuning retrieval, create a set of representative questions with expected evidence. This is often called a gold or labeled evaluation set.

Minimum Evaluation Record

```
{
  "query": "How many vacation days can carry over?",
  "relevant_chunk_ids": ["handbook_vacation_04"],
  "expected_answer": "Up to 5 days",
  "metadata_scope": {"region": "US", "version": "2026"}
}
```

What Makes a Good Dataset?

- Real user phrasing, including abbreviations and ambiguity.
- Easy, medium, and difficult questions.
- Questions requiring one chunk and questions requiring multiple chunks.
- Questions where the answer does not exist.
- Version/region/department-specific questions.
- Exact identifiers and numbers alongside semantic questions.
- Known historical failure cases.

Golden Rule

Do not tune on only five hand-picked questions. Build a growing regression set from real traffic, support failures, and subject-matter-expert review.

5. Recall, Precision, and Hit Rate

Recall@K

Recall@K asks: of all evidence we labeled as relevant, how much appeared in the top K results?

$\text{Recall@K} = \text{relevant items retrieved in top K} / \text{total relevant items}$

If 2 chunks are relevant and top 5 retrieves both:

$\text{Recall@5} = 2 / 2 = 1.00$

If top 5 retrieves only one:

$\text{Recall@5} = 1 / 2 = 0.50$

Precision@K

Precision@K asks: of the K retrieved results, how many are relevant?

Top 5 contains 2 relevant chunks:

$\text{Precision@5} = 2 / 5 = 0.40$

Hit@K

For many question-answering tasks, we may ask only whether **at least one** required evidence item appears in top K.

$\text{Hit@3} = 1$ if at least one relevant item appears in top 3

$\text{Hit@3} = 0$ otherwise

Metric	Best for	Warning
Recall@K	Coverage of all required evidence	Can look good even with lots of noise
Precision@K	How clean the result set is	Can punish useful extra context
Hit@K	Simple evidence presence	Does not measure rank or completeness

6. Ranking Metrics: MRR and nDCG

Reciprocal Rank and MRR

When the first relevant result appears earlier, users and downstream LLMs benefit. Reciprocal rank rewards early placement.

If first relevant result is rank 1: $RR = 1/1 = 1.00$

If first relevant result is rank 2: $RR = 1/2 = 0.50$

If first relevant result is rank 5: $RR = 1/5 = 0.20$

MRR = mean reciprocal rank across the evaluation queries

nDCG - Conceptual Introduction

Normalized Discounted Cumulative Gain is useful when relevance is graded rather than binary. A perfect source passage may be more useful than a loosely related passage, and earlier ranks receive more credit.

Beginner Priority

For Day 19, master Hit@K, Recall@K, Precision@K, and MRR first. Add nDCG when your project uses graded relevance and needs more sophisticated ranking analysis.

Ranking Example

Rank	Chunk	Relevance
1	General PTO overview	Partial
2	Vacation carryover clause	Exact
3	Holiday schedule	None
4	Vacation request process	Related

7. Top-K and Score Thresholds

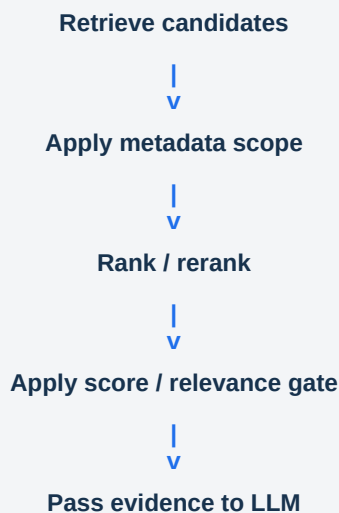
Top-K controls how many candidate results are returned or passed onward. More results can improve recall but also introduce noise, cost, and context dilution.

Small K	Large K
• Less context • Lower latency • Potentially higher precision • Risk: miss secondary evidence	• Higher candidate recall • Useful for multi-part questions • More reranking/context work • Risk: irrelevant noise

Score Threshold

A score threshold can reject low-scoring candidates. Thresholds are not universal truth boundaries; they depend on the embedding/retrieval system, corpus, query distribution, and ranking stage.

Candidate selection pipeline



How to Tune K and Threshold

- Start with a labeled dataset.
- Sweep several K values such as 3, 5, 10, 20.
- Track recall, precision, latency, and context size.
- Test threshold values on both answerable and unanswerable questions.
- Prefer the smallest candidate set that still meets your coverage target.

8. Metadata Filtering and Access Scope

Semantic similarity alone should not decide which content is eligible. Metadata filtering can restrict retrieval before ranking.

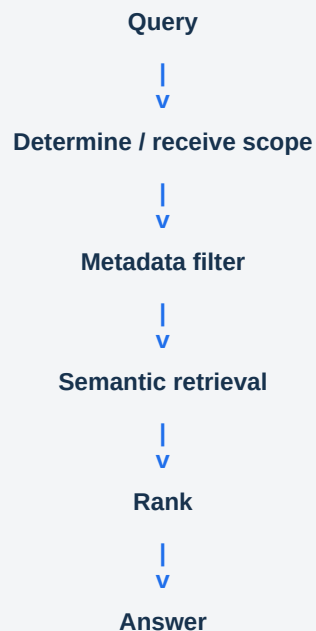
Useful Metadata

```
{
  "department": "HR",
  "region": "US",
  "version": 2026,
  "status": "active",
  "access_group": "employees",
  "document_type": "policy"
}
```

Example

Question: "What is the parental leave policy for US employees?" A high-quality system should avoid ranking a UK policy above the US policy simply because the wording is semantically similar.

Filter before generation



Security Principle

Permission filtering belongs in trusted application/retrieval logic. Do not rely on the LLM to politely ignore unauthorized chunks after they have already been retrieved.

9. Query Rewriting and Decomposition

Why Rewrite Queries?

User language may be vague, conversational, misspelled, or full of references. Query rewriting can create a clearer retrieval query while preserving the original intent.

User: "Can I move unused days to next year?"
Rewritten retrieval query: "vacation PTO unused days carryover next calendar year"

Query Decomposition

Some questions contain multiple information needs.

User: "Can I return the laptop, and how long will the refund take?"

Subquery 1: laptop return eligibility
Subquery 2: refund processing timeline

Risks

- Rewrite changes the user's meaning.
- Important exact identifier is removed.
- One broad query becomes an overly narrow query.
- Decomposition creates unnecessary tool/retrieval calls.

Evaluation Rule

Treat query rewriting as another component to evaluate. Compare retrieval with rewrite on vs off against the same gold dataset.

10. Hybrid Retrieval

Semantic retrieval is strong for meaning, but exact lexical matching can be better for product codes, error messages, legal clause numbers, names, and rare terminology. **Hybrid retrieval** combines semantic and lexical signals.

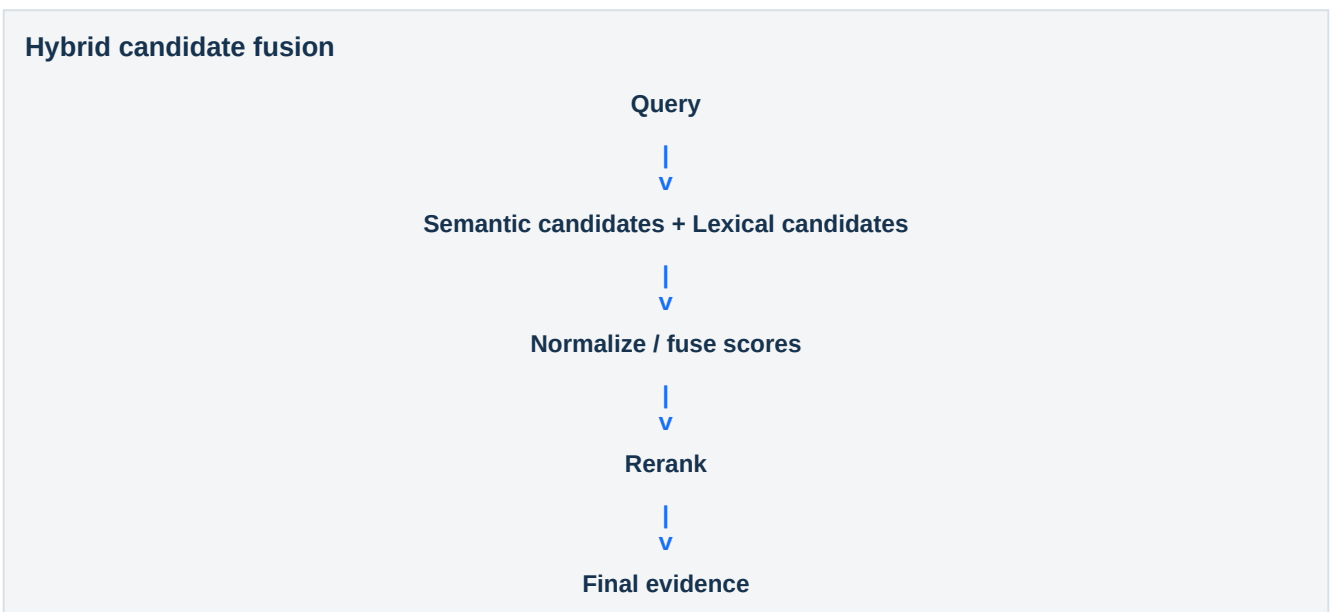
Semantic Strengths	Lexical Strengths
• Paraphrases • Concept similarity • Natural-language questions • Vocabulary mismatch tolerance	• Exact codes and names • Rare keywords • Part numbers • Exact phrases

Example

Query: "ERR-4921 payment timeout"

Semantic search may retrieve general payment-timeout guidance.
Keyword search can strongly match ERR-4921.
Hybrid ranking can combine both signals.

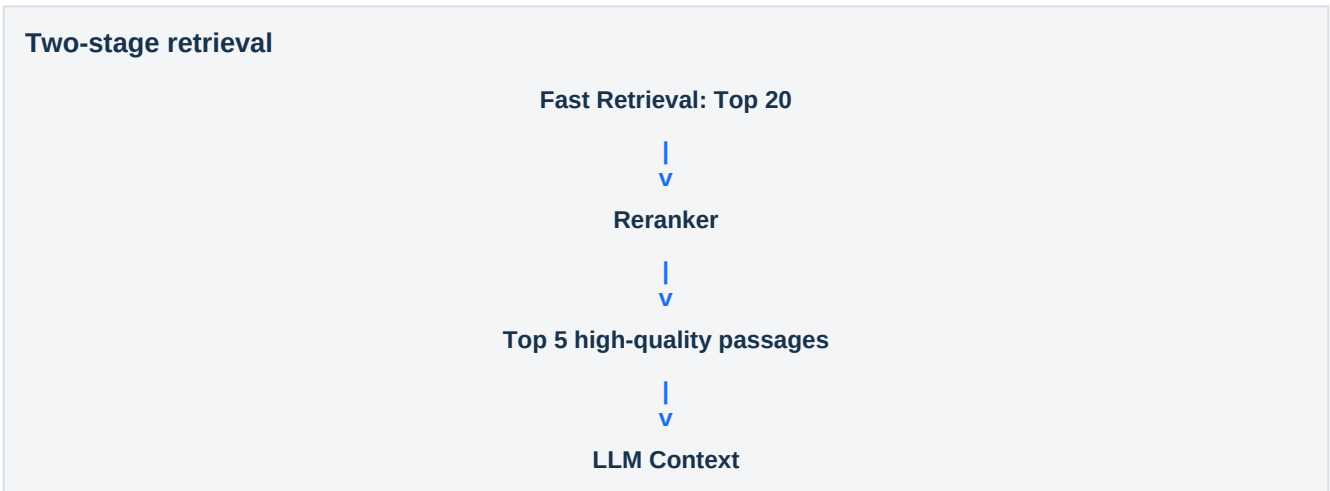
Hybrid Search Architecture



Hybrid search is a design pattern; implementations vary across vector databases and search engines. Measure it against your own query mix.

11. Reranking

Initial retrieval is often optimized for speed and candidate recall. A **reranker** can then reorder a smaller candidate set using a more expensive relevance model or ranking process.



Why Rerank?

- Improve ordering of semantically similar candidates.
- Reduce noisy passages before generation.
- Handle richer query-passage relevance than a single vector similarity score.
- Preserve fast first-stage search while spending more compute only on a smaller set.

Reranking Evaluation

Experiment	Recall@10	MRR	Answer accuracy
Baseline vector search	0.91	0.62	78%
+ reranker	0.91	0.79	86%

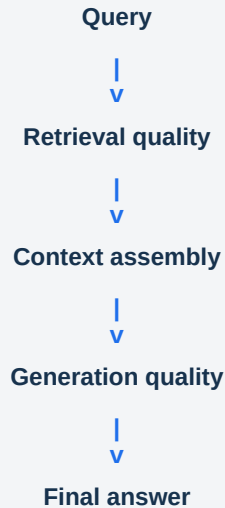
Illustrative numbers only. The key lesson is that reranking may improve **ordering** without changing candidate recall.

12. Answer-Level RAG Evaluation

Retrieval metrics are necessary but not sufficient. The final RAG system can still fail after retrieving good evidence.

Separate the Stages

Evaluate each stage separately



Answer-Level Dimensions

- Correctness - is the answer factually right?
- Groundedness - is every material claim supported by supplied evidence?
- Completeness - did the answer cover all required parts?
- Citation quality - do citations point to the passages that actually support the claims?
- Abstention - does the model avoid unsupported answers when evidence is missing?

Diagnostic Matrix

Retrieval	Answer	Likely problem
Good	Bad	Prompt/generation/context assembly
Bad	Bad	Retrieval/index/chunk/query
Good	Good	Healthy path
Bad	Good-looking	Possible hallucination / lucky prior knowledge

Professional Rule

Do not "fix the prompt" when the correct evidence never reached the model. Diagnose the earliest failing stage.

13. No-Answer and Negative Tests

A reliable knowledge assistant must know when the corpus does **not** contain enough evidence. Negative tests are essential for threshold tuning and hallucination prevention.

Examples

- Ask about a policy that does not exist.
- Ask about a future version not yet published.
- Ask for another region when only US documents are indexed.
- Ask for live account/order data that belongs in an API, not the knowledge base.
- Ask a deliberately misleading question containing unsupported assumptions.

Expected Behavior

Retrieved evidence: none above acceptance threshold

Expected answer:

"I do not have enough information in the approved knowledge sources to answer that.
Please check the current HR policy or contact HR."

Why Negative Tests Matter

A threshold that maximizes recall on answerable questions may also return plausible-looking junk for unanswerable questions. Evaluate both positive and negative cases.

14. Retrieval Failure Taxonomy

Classify failures before changing parameters. A good error taxonomy makes debugging systematic.

Failure type	Symptom	Likely fix
Parsing	Expected text never appears in index	Fix OCR/layout/parser
Chunking	Fact split or mixed with unrelated content	Change boundaries/context
Embedding/retrieval	Semantically relevant chunk missed	Embedding/query/candidate search
Metadata	Wrong region/version retrieved	Add/correct filters and attributes
Ranking	Relevant item found but ranked too low	Reranking/query treatment
Threshold	Good evidence discarded or junk accepted	Tune on labeled data
Context assembly	Correct passage truncated/duplicated	Deduplicate/expand/order context
Generation	Evidence present but answer wrong	Prompt/model/answer eval

Failure Review Template

Query: _____
Expected evidence: _____
Retrieved top K: _____
First relevant rank: _____
Failure category: _____
Hypothesis: _____
Change tested: _____
Metric before/after: _____

15. Tuning and Regression Workflow

RAG tuning loop



Change One Variable at a Time

- Chunk size/overlap
- Embedding model
- Top-K
- Metadata filtering
- Query rewrite
- Lexical + semantic hybrid
- Reranker
- Score threshold
- Context expansion

Ablation Thinking

If you turn off query rewriting and quality stays the same, the rewrite stage may not be earning its complexity. If reranking improves MRR but not final answers, investigate whether the generator already receives enough evidence.

Regression Tests

Every production failure should become a future test case. This prevents a new tuning change from silently reintroducing an old problem.

Optimization Goal

Optimize the full tradeoff: **quality + latency + cost + safety**. Highest recall at any cost is not automatically the best production system.

16. Current OpenAI Vector Store Search Controls

As of August 18, 2026, OpenAI Vector Store search exposes direct controls useful for Day 19 experimentation: query strings, file-attribute filters, maximum result count, ranking options, a score threshold, and optional query rewriting.

Current Search Parameters

Control	Current behavior
query	String or array of strings
filters	Comparison or compound filters over file attributes
max_num_results	Maximum results; current API reference specifies 1 to 50
ranking_options.ranker	none, auto, or current named default ranker
ranking_options.score_threshold	0 to 1
rewrite_query	Optional natural-language query rewriting

Python Search Example

```
from openai import OpenAI

client = OpenAI()

results = client.vector_stores.search(
    vector_store_id="vs_abc123",
    query="What is the US vacation carryover policy?",
    filters={
        "type": "and",
        "filters": [
            {"type": "eq", "key": "region", "value": "US"},
            {"type": "eq", "key": "status", "value": "active"},
        ],
    },
    max_num_results=8,
    ranking_options={
        "ranker": "auto",
        "score_threshold": 0.35,
    },
    rewrite_query=True,
)

for item in results.data:
    print(item.filename, item.score)
    for content in item.content:
        print(content.text)
```

Current API Note

OpenAI documents reranking as part of vector-store search ranking options, and notes that setting the ranker to **none** disables reranking and can reduce latency. Treat exact API values as version-sensitive and verify current docs when implementing.

17. Practical Python Evaluation Lab

This lab evaluates a tiny in-memory ranked result set. The same metric functions can later be applied to OpenAI vector-store results or another retriever.

Evaluation Dataset

```
EVALS = [
    {
        "query": "vacation carryover",
        "relevant": {"c2"},
    },
    {
        "query": "expense receipt threshold",
        "relevant": {"c5"},
    },
    {
        "query": "parental leave duration",
        "relevant": {"c8", "c9"},
    },
]
```

Metric Functions

```
def hit_at_k(ranked_ids, relevant, k):
    return int(bool(set(ranked_ids[:k]) & set(relevant)))

def precision_at_k(ranked_ids, relevant, k):
    top = ranked_ids[:k]
    if not top:
        return 0.0
    return sum(x in relevant for x in top) / len(top)

def recall_at_k(ranked_ids, relevant, k):
    if not relevant:
        return 1.0
    return sum(x in relevant for x in ranked_ids[:k]) / len(relevant)

def reciprocal_rank(ranked_ids, relevant):
    for i, item_id in enumerate(ranked_ids, start=1):
        if item_id in relevant:
            return 1.0 / i
    return 0.0
```

Evaluate a Retriever

```
def evaluate(eval_rows, retrieve, k=5):
    hits = []
    precisions = []
    recalls = []
    reciprocal_ranks = []

    for row in eval_rows:
        ranked_ids = retrieve(row["query"])
        relevant = row["relevant"]

        hits.append(hit_at_k(ranked_ids, relevant, k))
        precisions.append(precision_at_k(ranked_ids, relevant, k))
        recalls.append(recall_at_k(ranked_ids, relevant, k))
        reciprocal_ranks.append(reciprocal_rank(ranked_ids, relevant))

    n = len(eval_rows)
    return {
        f"hit@{k}": sum(hits) / n,
        f"precision@{k}": sum(precisions) / n,
        f"recall@{k}": sum(recalls) / n,
        "mrr": sum(reciprocal_ranks) / n,
    }
```

Threshold Sweep

```
for threshold in [0.1, 0.2, 0.3, 0.4, 0.5]:
    metrics = evaluate(
        EVALS,
        lambda q: retrieve(q, threshold=threshold),
        k=5,
    )
    print(threshold, metrics)
```

Lab Goal

Students should compare at least two configurations, record retrieval metrics, inspect individual failures, and write a recommendation. Never select a threshold from the aggregate score alone; inspect difficult and no-answer cases.

18. Production Monitoring and Business Design

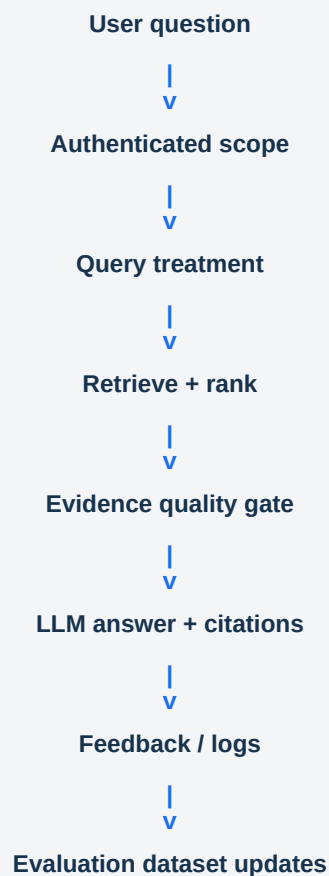
Offline evals are the starting point. Production RAG systems should also capture safe operational signals so quality changes can be detected after deployment.

Useful Monitoring Signals

- Query volume and latency.
- Number of retrieved results and score distribution.
- No-result / below-threshold rate.
- Which documents and versions are cited.
- User feedback or escalation rate.
- Repeated reformulations of the same question.
- Retrieval cost and generation token usage.
- Permission/filter failures.
- Known regression-query success rate after index updates.

Production Architecture

Continuous quality loop



Business Use Cases and Quality Targets

Use case	Retrieval priority
Employee policy assistant	Correct region/version + strong citations
Technical support	Exact errors plus semantic troubleshooting
Sales knowledge	Current product/pricing version
Legal/compliance	Clause-level evidence and strict access
Customer support	High recall with safe no-answer behavior

Client Discovery Questions

- What percentage of questions currently receive wrong or unsupported answers?
- Do you have a labeled set of real questions and source passages?
- Which errors are most costly: missing evidence or noisy evidence?
- Which documents have versions, regions, departments, or permissions?
- How often does the corpus change?
- What latency and cost targets exist?
- What should the system do when evidence is weak or missing?

19. Exercise, Quiz, Homework, and Interview Prep

Class Exercise A - Rank the Results

Query: "How many vacation days can carry over?"

- A. Holiday calendar
- B. Up to 5 unused vacation days may carry over
- C. Vacation requests require manager approval
- D. Expense claims must be filed within 30 days

Expected ranking: B first. C may be related but does not answer the question.

Class Exercise B - Diagnose

Retriever returns the exact vacation clause at rank 8, but top 5 are general PTO pages. Ask students: Is this primarily a recall problem, a candidate-size problem, or a ranking problem? **Likely ranking/candidate cutoff** if the relevant passage exists at rank 8.

10-Question Quiz

1. What does Recall@K measure?

Answer: How much of the labeled relevant evidence appears in the top K results.

2. What does Precision@K measure?

Answer: What fraction of the top K returned items are relevant.

3. What does MRR reward?

Answer: Placing the first relevant result near the top.

4. Why can a large Top-K be harmful?

Answer: It can add irrelevant context, cost, latency, and distract generation.

5. Why use metadata filters?

Answer: To restrict retrieval by scope such as region, version, department, or permissions.

6. What is query rewriting?

Answer: Transforming a user query into a retrieval-oriented query while preserving intent.

7. What is hybrid retrieval?

Answer: Combining semantic and lexical retrieval signals.

8. What is reranking?

Answer: Reordering an initial candidate set using a stronger relevance stage.

9. Why test no-answer questions?

Answer: To verify the system rejects weak evidence and avoids hallucination.

10. Why separate retrieval eval from answer eval?

Answer: Because the generator cannot reliably answer from evidence that retrieval never supplied.

Homework Assignments

- Create at least 30 realistic questions for one knowledge base and label the expected evidence chunk(s).
- Compute Hit@3, Recall@5, Precision@5, and MRR for your current retriever.
- Run Top-K values 3, 5, 10, and 20. Record quality, latency, and context-size changes.
- Create at least 8 no-answer questions and evaluate threshold behavior.
- Add metadata filters for version, region, and status. Compare wrong-version retrieval before and after.
- Compare query rewriting on vs off for at least 10 difficult questions.
- If possible, compare semantic-only vs hybrid retrieval on identifiers, codes, and natural-language questions.
- Create a failure report with at least 10 cases categorized as parsing, chunking, retrieval, filtering, ranking, threshold, context, or generation.
- Add every fixed failure to a regression dataset and rerun the full set after changes.

Interview Questions

What is retrieval evaluation?

Measuring whether a retrieval system returns the right evidence with useful ranking and acceptable noise.

Recall vs precision?

Recall measures coverage of relevant evidence; precision measures how clean the returned set is.

What is MRR?

Mean reciprocal rank, which rewards the first relevant result appearing early.

How do you choose Top-K?

Tune it on representative data using recall, precision, latency, context size, and answer outcomes.

What is a score threshold?

A relevance/similarity gate used to reject weak candidates; it must be calibrated on the project corpus.

Why metadata filters?

They improve relevance and enforce document scope such as permissions, region, version, and status.

Why use hybrid retrieval?

Exact lexical and semantic signals solve different query types and can complement each other.

Why rerank?

A fast first-stage retriever may have good candidate recall but poor ordering; reranking improves final relevance ordering.

How do you debug RAG?

Separate ingestion, chunking, retrieval, ranking/filtering, context assembly, and generation, then locate the earliest failing stage.

What are negative evals?

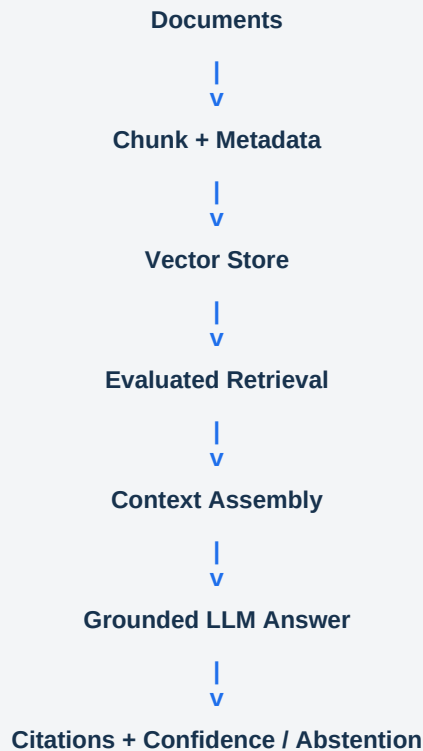
Questions intentionally lacking valid evidence, used to test abstention and weak-result rejection.

20. Day 20 Preview and References

Day 20 Preview - Building a Knowledge Assistant

Day 20 combines Days 15-19 into a complete RAG application: ingestion, vector storage, retrieval, context assembly, grounded answering, citations, evaluation, and business-facing architecture.

Day 20 end-to-end RAG assistant



The mini project will turn the retrieval lab into a practical Company Knowledge Assistant with a clear system prompt, evidence formatting, source citations, no-answer behavior, testing, and portfolio presentation.

Day 19 Final Takeaway

A RAG system is not reliable because it retrieves text. It becomes reliable when you can **define expected evidence, measure retrieval behavior, diagnose failures, and prove that tuning improves a regression set.**

Official OpenAI References

Current OpenAI-specific search controls were verified on August 18, 2026.

- OpenAI API Reference - Vector Store Search: https://developers.openai.com/api/reference/resources/vector_stores/methods/search
- OpenAI Developers - Working with evals: <https://developers.openai.com/api/docs/guides/evals>
- OpenAI Developers - Retrieval: <https://developers.openai.com/api/docs/guides/retrieval>

Further Study

Ranking evaluation, graded relevance, nDCG, reranker evaluation, query classification, retrieval tracing, online A/B testing, answer-groundedness graders, and continuous knowledge-base quality monitoring.